

```

'''Q1. An automobile company wants to predict a car's mpg value from its
(a) Load the dataset auto_mpg.csv and remove missing values.
(b) Identify predictor and target variables.
(c) Perform data splitting (80% train, 20% test).
(d) Fit a Linear Regression model and predict test outcomes.
(e) Evaluate the model using Mean Squared Error and R2 score.
(f) Discuss: If the R2 score = 0.85, what does it imply about model perf
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
df = pd.read_csv("auto_mpg.csv", na_values='')
df = df.dropna()
X = df.drop(['mpg'], axis=1)
y = df['mpg']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=123)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R2 Score: {r2:.4f}")

'''Q2. Exploring random sampling methods to estimate model uncertainty
(a) Load the btissue.csv dataset and extract only the feature columns.
(b) Use the resample() method to create a bootstrap sample of 100 obser
(c) Display the first 10 rows of the sample and check if any rows are r
import pandas as pd
from sklearn.utils import resample
df = pd.read_csv("btissue.csv")
X = df.drop(columns=["class"])
boot = resample(X, n_samples=100, replace=True, random_state=123)
print(boot.head(10))
print(boot.duplicated().any())

'''Q3. Instead of relying on a single train-test split, you want to check how consistent
(a) Use the btissue.csv dataset to implement 5-fold cross-validation.
(b) For each fold, print the train/test indices and record the number of samples
    used for training and testing.
(c) Summarize or visualize how the different folds cover the entire dataset without
    overlap.'''
import pandas as pd
from sklearn.model_selection import KFold
import numpy as np
df = pd.read_csv("btissue.csv")
kf = KFold(n_splits=5, shuffle=True, random_state=123)
all_test = []
for i, (train_idx, test_idx) in enumerate(kf.split(df)):
    print("Fold", i+1)
    print("Train indices", train_idx)
    print("Test indices", test_idx)
    print("Train size", len(train_idx), "Test size", len(test_idx))
    print()
    all_test.append(test_idx)
all_test = np.concatenate(all_test)
print("Total samples", len(df))
print("Total test indices collected across folds:", len(all_test))
print("Unique test indices", len(np.unique(all_test)))

'''Q4. Testing two validation techniques to measure model generalization tasks:
(a) Use the btissue.csv dataset and apply a Decision Tree Classifier.
(b) Evaluate model performance using:
    i) Holdout method (80/20 split)
    ii) 5-Fold Cross-Validation
(c) Compare the accuracy results from both methods.'''
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv("btissue.csv")
X = df.drop("class", axis=1)
y = df["class"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
holdout_acc = accuracy_score(y_test, dt.predict(X_test))
kf = KFold(n_splits=5, shuffle=True, random_state=123)
cv_acc = cross_val_score(dt, X, y, cv=kf).mean()
print("HOLDOUT ACCURACY:", holdout_acc)
print("5-Fold CV Accuracy", cv_acc)

'''Q5. Comparing model performance using all features vs. a subset (Iris dataset)
(a) Load the Iris dataset from sklearn.datasets.
(b) Create a DataFrame and display the first few rows.
(c) Train a Decision Tree Classifier using all features and record its accuracy.
(d) Select a subset of features (e.g., first two: sepal length and sepal width)
    using .iloc.
(e) Train another model using only the selected features and evaluate its accuracy
(f) Compare and discuss the results of both models.'''
from sklearn.datasets import load_iris
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
print(df.head())
X = df.iloc[:, :-1]
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42)

dt_all = DecisionTreeClassifier()
dt_all.fit(X_train, y_train)
acc_all = accuracy_score(y_test, dt_all.predict(X_test))
X_sub = df.iloc[:, :2]
X_train2, X_test2, y_train2, y_test2 = train_test_split(X_sub, y,
                                                         test_size=0.2,
                                                         random_state=42)

dt_sub = DecisionTreeClassifier()
dt_sub.fit(X_train2, y_train2)
acc_sub = accuracy_score(y_test2, dt_sub.predict(X_test2))
print("Accuracy with all features:", acc_all)
print("Accuracy with subset features:", acc_sub)

'''Q6. Comparing model performance using all features vs. a subset (Iris dataset)
(a) Load the Iris dataset using sklearn.datasets.
(b) Apply Principal Component Analysis (PCA) to reduce the
    four numerical features to two principal components.
(c) Create a DataFrame containing the two principal components
    and the target labels.
(d) Plot a 2D scatter plot of the components, using different colors
    for each class.'''
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import pandas as pd
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data
y = iris.target
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
df["target"] = y
print(pca.explained_variance_ratio_)
plt.scatter(df["PC1"], df["PC2"], c=df["target"])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

'''Q7. Dimensionality Reduction and Visualization using PCA (Iris Dataset)
(a) Load the Iris dataset using sklearn.datasets.
(b) Apply Principal Component Analysis (PCA) to reduce the
    four numerical features to two principal components.
(c) Create a DataFrame containing the two principal components
    and the target labels.
(d) Plot a 2D scatter plot of the components, using different colors
    for each class.'''
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import pandas as pd
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data
y = iris.target
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
df["target"] = y
print(pca.explained_variance_ratio_)
plt.scatter(df["PC1"], df["PC2"], c=df["target"])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

```

```
'''Q8. Encoding Categorical Data for Machine Learning (Employee Dataset)
(a) Create a DataFrame with the following columns and sample data:
Department (e.g., HR, IT, Finance)
Job_Role (e.g., Manager, Analyst, Clerk)
Marital_Status (e.g., Single, Married, Divorced)
(b) Display the original dataset.
(c) Encode categorical columns using:
Label Encoding for ordered or binary categories
One-Hot Encoding for nominal categories'''
```

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
df = pd.DataFrame({
    "Department": ["HR", "IT", "Finance", "IT"],
    "Job_Role": ["Manager", "Analyst", "Clerk", "Manager"],
    "Marital_Status": ["Single", "Married", "Divorced", "Single"]
})
print(df)
le = LabelEncoder()
df["Marital_Status_LE"] = le.fit_transform(df["Marital_Status"])
df_encoded = pd.get_dummies(df, columns=["Department", "Job_Role"])
print(df_encoded)
```

```
'''Q2. Write a Python program to generate random samples from a Normal
its Probability Density Function (PDF).'''
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
data = np.random.normal(0, 1, 1000)
x = np.linspace(-4, 4, 100)
pdf = norm.pdf(x, 0, 1)
plt.hist(data, bins=30, density=True)
plt.plot(x, pdf)
plt.title("Normal Distribution PDF")
plt.xlabel("Value")
plt.ylabel("Probability")
plt.show()
```

```
'''3. Write a Python program to compute and display the
covariance and correlation coefficient between two random variables'''
```

```
import numpy as np
X = np.random.rand(100)
Y = np.random.rand(100)
covariance = np.cov(X, Y)
correlation = np.corrcoef(X, Y)
print("Covariance Matrix:\n", covariance)
print("\nCorrelation Coefficient Matrix:\n", correlation)
```

```
'''Q1. Write a Python program to generate Bernoulli
and Binomial distributions using NumPy
and calculate their mean and variance.'''
```

```
import numpy as np
p = 0.6
bernoulli_data = np.random.binomial(1,p,1000)
print("Bernoulli Mean", np.mean(bernoulli_data))
print("Bernoulli Variance", np.var(bernoulli_data))
```

```
'''4. Write a Python program to verify the Central Limit Theorem
(CLT) by plotting the distribution of sample means.'''
```

```
import numpy as np
import matplotlib.pyplot as plt
population = np.random.uniform(0, 100, 10000)
sample_means = []

for i in range(1000):
    sample = np.random.choice(population, size=50)
    sample_means.append(np.mean(sample))
plt.hist(sample_means, bins=30)
plt.title("Central Limit Theorem Verification")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.show()
```

```
'''Q5. Write a Python program to implement kNN classifier for
given dataset. (apndcts.csv, btissue.csv)'''
```

The program should:

- Load a dataset from a CSV file using Pandas
- Separate predictor variables and target class labels
- Split the dataset into training and testing sets
- Train a kNN model using the training data
- Predict class labels for the test data
- Compute and display the accuracy of the model'''

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("iris.csv")
# Drop ID column if present
data = data.drop(columns=['Id'], errors='ignore')
# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# kNN model
knn = KNeighborsClassifier(n_neighbors=5)
# Train model
knn.fit(X_train, y_train)
# Prediction
y_pred = knn.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("kNN Accuracy:", accuracy)
```

"""Case Study 1 - Student Performance Analytics
 You are given data of 100 students containing their marks in Math, Science, and
 (a) Using Pandas, calculate the average marks for each student and identify the
 (b) Compute subject-wise average and standard deviation; comment which subject
 (c) Draw a boxplot for each subject and comment on outliers."""

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
data = {
    'StudentID': [f'Student_{i+1}' for i in range(100)],
    'Math': np.random.randint(40, 101, size=100),
    'Science': np.random.randint(30, 101, size=100),
    'English': np.random.randint(35, 101, size=100)
}
df = pd.DataFrame(data)
print(df.head())
df['Average'] = df[['Math', 'Science', 'English']].mean(axis=1)
top_5 = df.sort_values(by='Average', ascending=False).head(5)
print("Top 5 Performers:")
print(top_5[['StudentID', 'Average']])
subject_stats = df[['Math', 'Science', 'English']].agg(['mean', 'std'])
print("\nSubject-wise statistics:\n", subject_stats)
highest_variation_subject = subject_stats.loc['std'].idxmax()
print(f"\nSubject with highest variation: {highest_variation_subject}")
plt.figure(figsize=(10, 6))
sns.boxplot(data=df[['Math', 'Science', 'English']])
plt.title('Boxplot of Marks by Subject')
plt.show()
```

"""Case Study 2 - Sales Data Exploration
 A company records sales transactions in a dataset with columns: Product, Quantity
 (a) Add a new column Total = Quantity × Price and compute overall revenue.
 (b) Group data by region and identify which region contributes the most sales.
 (c) Plot a histogram of total sales per product and discuss which products are u

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
products = ['Laptop', 'Phone', 'Tablet', 'Monitor', 'Keyboard']
regions = ['North', 'South', 'East', 'West']
data = {
    'Product': np.random.choice(products, size=100),
    'Quantity': np.random.randint(1, 20, size=100),
    'Price': np.round(np.random.uniform(50, 1000, size=100), 2),
    'Region': np.random.choice(regions, size=100)
}
df = pd.DataFrame(data)
print(df.head())
df['Total'] = df['Quantity'] * df['Price']
overall_revenue = df['Total'].sum()
print(f"Overall Revenue: ${overall_revenue:,.2f}")
region_sales = df.groupby('Region')['Total'].sum().sort_values(ascending=False)
top_region = region_sales.idxmax()
print("\nSales by Region:")
print(region_sales)
print(f"\nRegion with highest sales: {top_region}")
product_sales_df = product_sales.reset_index()
product_sales_df.columns = ['Product', 'TotalSales']
plt.figure(figsize=(8, 5))
sns.barplot(data=product_sales_df, x='Product', y='TotalSales')
plt.title('Total Sales by Product')
plt.ylabel('Total Sales ($)')
plt.xlabel('Product')
plt.xticks(rotation=45)
plt.show()
print("\nProducts with lowest sales might be considered underperforming.")
print(product_sales)
```

"""Case Study 3 - Predicting Fuel Efficiency (Auto MPG Dataset)
 You are analyzing the Auto MPG dataset (UCI Repository) with features: mpg, cyli
 weight, acceleration, model year, origin.
 (a) Compute the correlation of mpg with each numeric feature and discuss which f
 (b) Plot a scatter plot of weight vs mpg; describe the trend you observe."""

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset('mpg').dropna()
print(df.head())
numeric_features = df.select_dtypes(include='number').columns.drop('mpg')
correlations = df[numeric_features].corrwith(df['mpg'])
print("Correlation of mpg with each numeric feature:")
print(correlations.sort_values(ascending=False))
plt.figure(figsize=(8, 6))
sns.scatterplot(x='weight', y='mpg', data=df)
plt.title('Scatter Plot of Weight vs MPG')
plt.xlabel('Weight')
plt.ylabel('MPG')
plt.show()
```

```
# --- Decision Tree Model ---
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt_model = dt.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
# --- Evaluation Metrics ---
print("\n--- Decision Tree Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Feature importances:", dt.feature_importances_)
print("Precision:", precision_score(y_test, y_pred_dt, average='macro', zero_division=0))
print("Recall:", recall_score(y_test, y_pred_dt, average='macro', zero_division=0))
print("F1-score:", f1_score(y_test, y_pred_dt, average='macro', zero_division=0))
```

```
# --- Confusion Matrix Visualization ---
cm = confusion_matrix(y_test, y_pred_dt)
plt.figure(figsize=(4, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score,
                                recall_score, f1_score, confusion_matrix

# --- Load Dataset ---
df = pd.read_csv("btissue.csv")
print("\n--- Dataset Preview ---")
print(df.head())
print("\n--- Dataset Info ---")
print(df.info())
print("\nMissing values per column:")
print(df.isnull().sum())
print("\n--- Summary Statistics ---")
print(df.describe())

# --- Feature and Target Extraction ---
X = df.iloc[:, :-1].values # Features
y = df.iloc[:, -1].values  # Target
print("\nFeature shape:", X.shape)
print("\nTarget shape:", y.shape)

# --- Train-Test Split ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

print("\nTrain shape:", X_train.shape)
print("\nTest shape:", X_test.shape)

# --- KNN Classification ---
knn = KNeighborsClassifier(n_neighbors=3)
model = knn.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\n--- KNN Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='macro', zero_division=0))
print("Recall:", recall_score(y_test, y_pred, average='macro', zero_division=0))
print("F1-score:", f1_score(y_test, y_pred, average='macro', zero_division=0))

# --- Confusion Matrix ---
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(4, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# --- Random Forest Model ---
rf = RandomForestClassifier(random_state=42)
rf_model = rf.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

# --- Evaluation Metrics ---
print("\n--- Random Forest Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Precision:", precision_score(y_test, y_pred_rf, average='macro', zero_division=0))
print("Recall:", recall_score(y_test, y_pred_rf, average='macro', zero_division=0))
print("F1-score:", f1_score(y_test, y_pred_rf, average='macro', zero_division=0))

# --- Confusion Matrix Visualization ---
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(4, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# --- SVM Model ---
svm = SVC(kernel='linear', random_state=42)
svm_model = svm.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)

# --- Evaluation Metrics ---
print("\n--- SVM Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Precision:", precision_score(y_test, y_pred_svm, average='macro', zero_division=0))
print("Recall:", recall_score(y_test, y_pred_svm, average='macro', zero_division=0))
print("F1-score:", f1_score(y_test, y_pred_svm, average='macro', zero_division=0))

# --- Confusion Matrix Visualization ---
cm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(4, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```